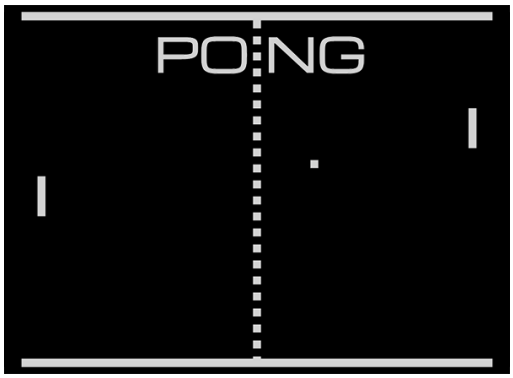
Assignment 2 – Event Handling Part2 – Pong Game

Goal

The aim of this assignment is to put into practice the object-oriented programming principles and techniques covered in class, including inheritance, polymorphism, and event-driven development. The goal is to further extend the Entity-Component framework for game development by adding support for event handling. As part of this exercise, students will implement a Pong game using the framework.

The resulting game should be a human-versus-CPU Pong game in which the Player entity can respond to input events, while the CPU can react to various events, such as collisions between the ball and the player's paddle or between the ball and the top and bottom boundaries of the playing field.



Description

This is an open-ended assignment that can be implemented in different ways, provided that the framework is followed. However, the game itself must remain very simple: the ball can move at a constant speed, and no physics simulation is required. When the ball reaches either the left or right side of the screen, meaning that either the player or the CPU loses, the game is over, and the game state must be reset.

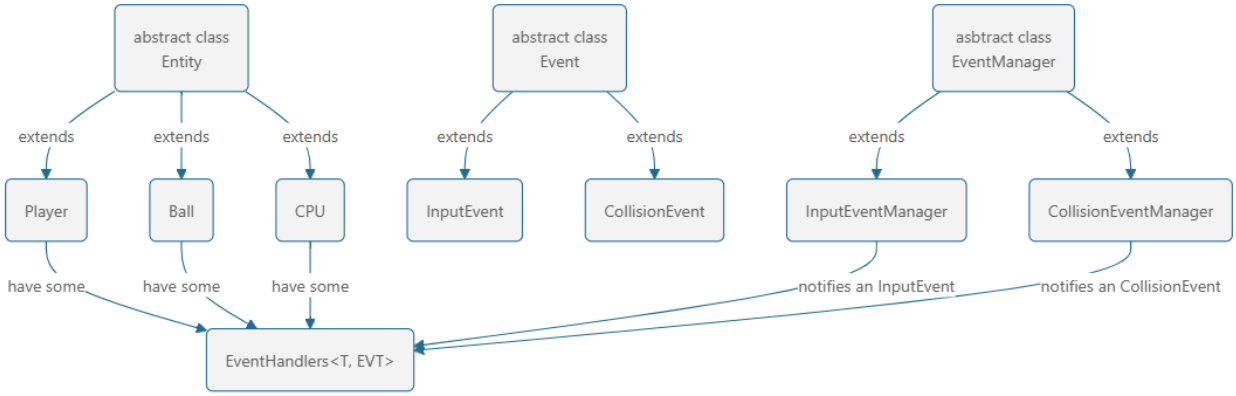
To control the Player, we will use an Event-Driven Development (**EDD**) approach. To move the paddle (or racket) up and down, the Player must respond to input events, such as key-press events. **Important:** In EDD, the entity interested in an event must be decoupled/disconnected from the entity that detects and triggers that event. The purpose of this separation is to allow the input-handling mechanism to be extended to support other input devices, such as a gamepad or a mouse, without requiring changes to the entity's logic to accommodate each device.

To follow the EDD principles, you can implement an **`InputEventManager`** class that captures keyboard input, creates custom events, and dispatches them to the appropriate subscribers. To implement the subscription mechanism, we will use a generic event-handling approach based on templates, in which each entity has a custom event handler as a property. These classes can be implemented by extending

the **`EventManager`** and **`EventHandler<T, EVT>`** classes, which are already provided in the framework folder.

The CPU's AI should also be very simple. For example, in the provided demo, the CPU starts moving when the ball collides with the player's paddle (or racket), following the ball's position along the Y-axis. The CPU stops moving when it hits the ball or when the ball collides with the top or bottom boundaries of the screen. Similarly, you can implement a `CollisionEventManager` class that extends `EventManager` to detect collisions between entities and notify the CPU by dispatching the appropriate events.

One possible class organization, which is also an ideal starting point, is shown below:



The **`Entity`** class is abstract and can be extended to implement specific game entities. It provides two new virtual functions, **`Update()`** and **`Draw()`**, which can be overridden to implement the entity's internal logic and render it, respectively. Similarly, the **`EventManager`** class is also abstract and provides a virtual **`Update()`** function, which can be overridden to handle tasks such as detecting keyboard input, detecting collisions between entities, creating events, and notifying the entities interested in those events through their specific Event Handlers. These methods can be called during the Update and Draw phases/states of the game loop within the Game State Machine.

These classes are provided in the framework folder and should NOT be modified.

What to do:

- The code must run without errors and the organization of the code, comments and adherence to the event driven development principles will be also evaluated.

Submission Guidelines

You have to submit all the cc, hh and cpp files in a zip folder on Moodle with the following naming conventions: cs225fa26_{username}_2-2.zip